

Rodrigo Fernandez

Senior React.js / .NET Full Stack Engineer

Katy, Texas | +1 (407) 801-1287 | www.linkedin.com/in/rodrigobermejo-fernandez-9a5a0664 | rodrigobfdev@gmail.com

Summary

Senior **React 16–18** / **.NET Core 3.1–.NET 8 Full Stack Engineer** with strong delivery history across **cloud hosting, fintech SaaS, healthcare portals, eCommerce, and enterprise workflow platforms**, building secure SPAs, high-throughput APIs, billing dashboards, identity modules, and reporting tools with **C# 8–12, ASP.NET Core 3.1–8, Entity Framework Core 3–8, TypeScript 4–5, Redux Toolkit, SQL Server 2016–2022, Azure DevOps, Docker**, and CI/CD practices that improved release quality, reduced production defects, and helped teams modernize legacy .NET Framework systems into scalable cloud-ready architecture for regulated customer environments and distributed engineering teams.

Skills

- **Frontend: React 16–18, TypeScript 4–5**, JavaScript ES6+, Redux, Redux Toolkit, React Query, Context API, Hooks, Material UI, Tailwind CSS, Bootstrap, HTML5, CSS3, accessibility fixes, reusable component architecture
- **Backend: C# 8–12, .NET Core 3.1–.NET 8, ASP.NET Core 3.1–8**, ASP.NET MVC 5, Web API 2, REST API design, JWT, OAuth2, RBAC, clean architecture, middleware, background jobs, API versioning
- **Database: SQL Server 2016–2022**, Entity Framework Core 3–8, LINQ, stored procedures, indexing, query optimization, migrations, transaction handling, reporting queries
- **Cloud & DevOps: Azure DevOps, GitHub Actions, Docker, Kubernetes basics**, Azure App Service, Application Insights, IIS, CI/CD pipelines, release approvals, feature flags, health checks, logging, monitoring
- **Testing & Quality: xUnit, NUnit, Moq, Jest, React Testing Library**, integration testing, code review, debugging, performance profiling, production incident resolution

Experience

Lightedge — Senior Software Engineer *(Apr 2020 – Mar 2026)*

- Delivered **React 17–18** customer portals with **TypeScript 4–5** and reusable component libraries, improving dashboard load consistency by **34%** across hosting, billing, and infrastructure workflows.
- Built secure **ASP.NET Core 6–8** APIs with **C# 10–12**, JWT authentication, role-based authorization, and structured logging for cloud service provisioning and customer account operations.
- Migrated legacy **.NET Framework 4.8** service modules into **.NET 6–8** microservices, reducing deployment friction by **42%** while preserving existing SQL Server business rules.
- Resolved production memory spikes caused by unbounded API payloads by adding pagination, response compression, query projections, and **EF Core 6–8** tracking optimizations.
- Implemented new self-service billing and usage features with **React Query, Redux Toolkit**, and **ASP.NET Core Web API**, reducing support ticket dependency by **29%**.
- Refactored shared React forms into schema-driven components with validation, feature flags, and accessibility fixes, cutting duplicate UI code by **38%** across customer workflows.
- Optimized **SQL Server 2019–2022** stored procedures, indexes, and EF queries for reporting endpoints, reducing several high-traffic report response times from seconds to sub-second ranges.

- Strengthened CI/CD pipelines in **Azure DevOps** and GitHub Actions with build validation, automated tests, Docker image checks, and release approvals for safer production deployments.
- Solved intermittent authentication failures by tracing token refresh race conditions in React interceptors and updating server-side session validation in **ASP.NET Core** middleware.
- Integrated observability with **Application Insights**, structured Serilog events, health checks, and alert rules, helping engineering detect API regressions before customers reported incidents.
- Coordinated flexible full-stack delivery across frontend, backend, database, and DevOps tasks, supporting roadmap work while quickly closing urgent production bugs and customer escalations.
- Mentored engineers on **React 18**, **C#**, clean architecture, API versioning, and pull request quality, raising team review consistency and reducing escaped defects by **26%**.

NIX United — Software Engineer *(Oct 2016 – Mar 2020)*

- Developed enterprise SaaS modules with **React 15–16**, **Redux 3–4**, **ASP.NET MVC 5**, and **Web API 2**, supporting finance, logistics, and healthcare client workflows.
- Upgraded selected backend services from **.NET Framework 4.6–4.8** into **.NET Core 2.1–3.1**, improving API maintainability while keeping legacy integrations stable.
- Implemented customer onboarding screens, admin permissions, and audit-history views with reusable React components and server-side validation in **C# 7–8** APIs.
- Fixed a recurring checkout issue caused by stale Redux state and duplicate API submissions by introducing idempotency keys, UI disable states, and stronger Web API validation.
- Improved SQL Server query performance by tuning indexes, reviewing execution plans, and replacing inefficient ORM calls, lowering slow transaction frequency by **31%**.
- Built migration adapters that allowed old MVC pages and newer React modules to coexist, helping teams modernize gradually without blocking active client releases.
- Added automated unit and integration tests with **xUnit**, **Moq**, and frontend test coverage, reducing regression defects by **24%** during high-change release cycles.
- Implemented role-based access controls, secure password reset flows, and audit logging for sensitive business modules used by healthcare and financial operations teams.
- Resolved browser compatibility bugs in React views by standardizing Babel configuration, polyfills, and CSS patterns for enterprise users still running controlled desktop environments.
- Collaborated with QA, product, and DevOps teams to diagnose production incidents, reproduce edge cases, and deliver hotfixes without creating risky database changes.
- Used **Azure DevOps**, Git branching standards, and code review checklists to keep distributed teams aligned and improve release predictability across several client projects.

Webiz Digital — Software Developer *(Jan 2014 – Sep 2016)*

- Built business web applications with **ASP.NET MVC 5**, **C# 5–6**, **Web API 2**, **jQuery 1.x–2.x**, **Bootstrap 3**, and **SQL Server 2012–2016**.
- Developed eCommerce admin panels, product catalog tools, order workflows, promotion rules, and reporting screens using MVC controllers and Razor views.
- Implemented secure REST endpoints with **Web API 2**, validation filters, error handling, and transaction-safe database updates for client-facing systems.
- Resolved payment callback failures caused by timeout handling and duplicate status updates by adding retry-safe logic, audit records, and clearer exception logs.
- Migrated older Web Forms-style admin pages into **ASP.NET MVC 5**, reducing duplicated code paths and improving maintainability by **27%**.
- Improved storefront performance by bundling scripts, reducing unnecessary DOM updates, optimizing images, and cleaning large jQuery event handlers.

- Created reusable validation helpers, shared layouts, and common UI components that made forms, messages, and error states consistent across projects.
- Supported production incidents by reviewing IIS logs, SQL traces, browser console errors, and customer reports before shipping controlled hotfix releases.
- Integrated email campaign tools, payment notifications, and shipping updates while protecting credentials through environment-based configuration practices.
- Worked flexibly across frontend, backend, database, and deployment support tasks, helping small teams deliver client features with stable **.NET** foundations.

Aurio — Junior Software Developer *(Oct 2012 – Dec 2013)*

- Supported internal workflow tools built with **ASP.NET MVC 4**, **C# 4–5**, **jQuery**, **Bootstrap 2–3**, and **SQL Server 2008 R2–2012** for operations and reporting teams.
- Built CRUD screens, approval flows, and export reports using Razor views, MVC controllers, ADO.NET patterns, and stored procedures under senior engineering guidance.
- Fixed form validation defects caused by inconsistent client-side rules by aligning jQuery validation, server model binding, and database constraint messages.
- Improved reporting accuracy by correcting SQL joins, date filters, and null-handling logic that previously caused mismatched totals in monthly operational dashboards.
- Helped migrate small legacy pages into MVC-based modules, reducing duplicated code paths and making future maintenance easier for the internal application team.
- Created reusable UI snippets for tables, filters, modals, and notification messages, giving business users more consistent behavior across daily workflow screens.
- Investigated IIS deployment issues, connection-string mistakes, and permission errors, documenting repeatable fixes so releases became less dependent on senior developers.
- Partnered with QA and business users to reproduce edge cases, verify fixes, and improve acceptance criteria before small releases were promoted to production.
- Built a strong foundation in **C#**, **SQL Server**, debugging, and clean web application structure while contributing dependable fixes across real production systems.

Microsoft — Software Developer Intern *(Jun 2012 – Sep 2012)*

- Contributed to internal engineering utilities using **C#**, **ASP.NET MVC 4**, JavaScript, and SQL Server patterns while learning enterprise development standards from senior engineers.
- Assisted with bug reproduction, log review, and small feature fixes for dashboard pages, improving reliability of internal tools used by technical support teams.
- Implemented minor UI updates and backend validation improvements under code review, gaining practical experience with accessibility, maintainability, and secure coding expectations.
- Prepared documentation for setup steps, known issues, and troubleshooting paths so future interns and team members could onboard more smoothly to internal applications.

Education

Texas State University

Bachelor's Degree in Computer Science *(Sep 2008 – May 2012)*